

Projeto Final

Laboratório de Bases de Dados - Sigla

Interface de controle e exploração de base de dados da Fórmula 1

Catarina Moreira Lima – 8957221

Nome 2 – NUSP

Nome 3 – NUSP

Junho de 2026

1 Introdução

Este trabalho apresenta o desenvolvimento de um protótipo de aplicação completa para exploração da base de dados da Fórmula 1 integrada a dados geográficos de cidades, países e aeroportos.

A aplicação permite autenticar usuários com perfis distintos, apresentar *dashboards* personalizados, executar relatórios e realizar operações de cadastro conforme as permissões de cada tipo de usuário.

2 Preparação da Base de Dados

A base utilizada foi preparada a partir dos *scripts* desenvolvidos ao longo da disciplina. A execução foi organizada na seguinte ordem:

1. `create_table.sql`: criação do esquema relacional;
2. `insert_table.sql`: carga dos arquivos CSV e TSV;
3. `clean_data.sql`: normalização, deduplicação e ajuste de vínculos entre as tabelas;
4. `app_users.sql`: usuários, autenticação, auditoria e triggers;
5. `app_actions.sql`: ações de cadastro e consulta por perfil;
6. `app_indexes.sql`: índices auxiliares;
7. `app_views.sql`: views reutilizáveis pela aplicação;
8. `app_dashboard.sql`: funções de *dashboard*;
9. `app_reports.sql`: funções de relatórios.

2.1 Execução com Docker

Para facilitar a reprodução do ambiente entre os membros do grupo, foi utilizado Docker com PostgreSQL. A carga foi separada em dois scripts: um para a base de dados e outro para a camada da aplicação.

```
./scripts/load_database.sh
./scripts/load_app.sh
```

O script `load_database.sh` executa a criação do esquema, a carga dos dados e a limpeza. O script `load_app.sh` executa os arquivos de usuários, ações, índices, views, dashboards e relatórios. Assim, durante o desenvolvimento de novas funções, é possível recarregar apenas a camada da aplicação.

Os dados de conexão utilizados foram:

- Banco: `f1db`
- Usuário: `f1user`
- Porta: `5433`

3 Views Reutilizáveis

Foi criado o arquivo `app_views.sql` para concentrar consultas reutilizáveis pela aplicação. A view `vw_resultados_corridas` reúne os dados de resultados de corrida com temporada, piloto, escuderia, status e circuito.

Essa view centraliza junções frequentes entre as tabelas `results`, `races`, `seasons`, `drivers`, `constructors`, `status` e `circuits`. Com isso, as funções de *dashboard* e de relatórios ficam mais simples e reutilizam a mesma base lógica. Os filtros específicos continuam sendo aplicados nas funções, de acordo com o perfil do usuário logado, como administrador, escuderia ou piloto.

Também foi criada a view `vw_aeroportos_cidades_paises`, que reúne aeroportos, cidades, países e tipos de aeroporto. Essa view é utilizada no relatório de aeroportos próximos a uma cidade brasileira, mantendo na função apenas o filtro pela cidade informada e o cálculo da distância geográfica.

A view `vw_corridas_resumo` resume cada corrida com ano, rodada, circuito, voltas registradas e quantidade de pilotos participantes. Ela é usada no relatório hierárquico de corridas do administrador, evitando repetir a agregação dentro da função de relatório.

A view `vw_escuderia_pilotos` reúne escuderias, pilotos, países e a origem do vínculo entre escuderia e piloto. Essa view é construída sobre a tabela `app_escuderia_pilotos`, que combina vínculos históricos inferidos de `results` com vínculos criados pela importação de pilotos por arquivo.

4 Modelo de Usuários e Autenticação

A aplicação possui três tipos de usuários, cada um com permissões e visões distintas:

- **Admin:** usuário administrador do sistema;
- **Escuderia:** usuário associado a uma escuderia;

- **Piloto:** usuário associado a um piloto.

Foi criada a tabela `users`, contendo os atributos `userid`, `login`, `password`, `tipo` e `id_original`. O atributo `id_original` referencia o identificador original de piloto ou escuderia, conforme o tipo de usuário.

4.1 Proteção das Senhas

A autenticação foi implementada diretamente pela tabela `users`. Por isso, as senhas não são armazenadas em texto puro. Foi utilizada a extensão `pgcrypto`, com as funções `crypt()` e `gen_salt()`. A função `app_alterar_senha` permite trocar a senha validando a senha atual do usuário.

```
CREATE OR REPLACE FUNCTION app_hash_password(p_plain_password TEXT)
RETURNS TEXT AS $$
BEGIN
    RETURN crypt(p_plain_password, gen_salt('bf'));
END;
$$ LANGUAGE plpgsql;
```

4.2 Auditoria de Acesso

Foi criada a tabela `users_log`, responsável por registrar eventos de `LOGIN` e `LOGOUT`. A função `app_login` registra automaticamente a entrada do usuário quando a autenticação é realizada com sucesso.

5 Triggers Implementadas

5.1 Sincronização de Pilotos com Usuários

A trigger associada à tabela `drivers` cria automaticamente um usuário do tipo `Piloto` sempre que um novo piloto é cadastrado. Em atualizações, a trigger ajusta o login quando necessário, mas preserva a senha já alterada pelo usuário.

5.2 Remoção de Usuários de Pilotos

Também foi criada uma trigger associada à tabela `drivers` para remover automaticamente o usuário do tipo `Piloto` quando o piloto correspondente é removido. Antes da remoção do usuário, os registros relacionados em `users_log` são apagados para respeitar a chave estrangeira existente entre `users_log` e `users`.

5.3 Sincronização de Escuderias com Usuários

A trigger associada à tabela `constructors` cria automaticamente um usuário do tipo `Escuderia` sempre que uma nova escuderia é cadastrada. Em atualizações, a trigger ajusta o login quando necessário, mas preserva a senha já alterada pelo usuário.

5.4 Remoção de Usuários de Escuderias

Foi criada uma trigger associada à tabela `constructors` para remover automaticamente o usuário do tipo `Escuderia` quando a escuderia correspondente é removida. Assim como no caso dos pilotos, os registros associados em `users_log` são removidos antes da exclusão do usuário.

5.5 Atualização de Data em Usuários

A trigger de atualização da tabela `users` atualiza automaticamente o campo `updated_at` quando uma tupla é modificada.

5.6 Sincronização de Vínculos entre Escuderias e Pilotos

A trigger associada à tabela `results` mantém a tabela `app_escuderia_pilotos` atualizada quando novos resultados criam vínculos entre pilotos e escuderias. Essa estrutura aceita que um mesmo piloto esteja vinculado a mais de uma escuderia, preservando mudanças de escuderia e histórico de carreira.

6 Ações Implementadas

Foram criadas funções auxiliares para representar ações solicitadas na especificação:

- `app_admin_cadastrar_escuderia`: cadastra uma nova escuderia e deixa a criação do usuário correspondente a cargo da trigger;
- `app_admin_cadastrar_piloto`: cadastra um novo piloto e deixa a criação do usuário correspondente a cargo da trigger;
- `app_escuderia_processar_importacao_pilotos`: processa linhas carregadas por arquivo em uma tabela de apoio, cadastrando pilotos novos e criando vínculos com a escuderia;
- `app_escuderia_listar_pilotos`: lista os pilotos vinculados à escuderia no cadastro da aplicação;
- `app_escuderia_consultar_piloto_por_sobrenome`: consulta pilotos pelo sobrenome, limitando os resultados aos pilotos vinculados à escuderia logada.

Para suportar a importação de pilotos por escuderia, foi criada uma tabela própria de vínculo entre escuderias e pilotos. A relação original entre piloto e escuderia aparece apenas em `results`; portanto, pilotos recém-importados ainda não teriam vínculo visível caso a aplicação dependesse somente dos resultados históricos. Por isso, a tabela é inicialmente populada com pares distintos de `results`, recebe vínculos criados por importação de arquivo e é atualizada por trigger quando novos resultados são registrados.

Após a revisão das funções, foi mantida a separação entre consultas de desempenho e consultas cadastrais. Dashboards e relatórios que calculam pontos, vitórias, temporadas, corridas ou status usam `results` ou `vw_resultados_corridas`, pois dependem de participação real em corridas. Já as consultas cadastrais de pilotos vinculados à escuderia usam `app_escuderia_pilotos` ou `vw_escuderia_pilotos`, permitindo incluir pilotos importados por arquivo mesmo antes de existirem resultados para eles.

7 Dashboards

7.1 Dashboard do Administrador

O dashboard do administrador apresenta:

- quantidade total de pilotos, escuderias e temporadas;
- corridas cadastradas na temporada mais recente;
- escuderias da temporada mais recente com total de pontos;
- pilotos da temporada mais recente com total de pontos.

7.2 Dashboard da Escuderia

O dashboard da escuderia apresenta:

- quantidade de vitórias da escuderia;
- quantidade de pilotos diferentes que já correram pela escuderia;
- primeiro e último ano com dados da escuderia.

7.3 Dashboard do Piloto

O dashboard do piloto apresenta:

- primeiro e último ano com dados do piloto;
- desempenho por ano e circuito, incluindo pontos, vitórias e corridas disputadas.

8 Relatórios

8.1 Relatório 1 – Admin: Resultados por Status

Este relatório apresenta a quantidade de resultados agrupados por status de corrida.

8.2 Relatório 2 – Admin: Aeroportos Próximos a Cidade Brasileira

Este relatório recebe o nome de uma cidade e retorna aeroportos brasileiros dos tipos `medium_airport` e `large_airport` localizados a até 100 km da cidade pesquisada.

Para o cálculo de distância foi utilizada a extensão `earthdistance`, juntamente com `cube`.

```
earth_distance(  
    ll_to_earth(c.latitude, c.longitude),  
    ll_to_earth(a.latitude_deg, a.longitude_deg)  
) / 1000.0
```

8.3 Relatório 3 – Admin: Relatório Hierárquico de Corridas

O relatório 3 foi dividido em duas partes:

- `app_admin_relatorio_3_escuderias_pilotos`: lista todas as escuderias cadastradas e a quantidade de pilotos que correram por cada uma;
- `app_admin_relatorio_3_hierarquico_corridas`: organiza as corridas em formato hierárquico.

A parte hierárquica foi organizada em três níveis:

1. total de corridas cadastradas;
2. estatísticas de corridas por circuito;
3. detalhes das corridas de cada circuito.

Para representar o relatório multinível em formato tabular, a função retorna uma coluna `nível`. Assim, as linhas de nível 1 representam o total geral, as linhas de nível 2 representam o resumo por circuito e as linhas de nível 3 representam as corridas pertencentes a cada circuito.

8.4 Relatório 4 – Escuderia: Vitórias por Piloto

Este relatório lista os pilotos que correram por uma escuderia e a quantidade de vezes em que cada piloto obteve primeira posição.

8.5 Relatório 5 – Escuderia: Resultados por Status

Este relatório apresenta a quantidade de resultados por status, limitada ao escopo da escuderia logada.

8.6 Relatório 6 – Piloto: Pontos por Ano e Corridas

Este relatório apresenta os pontos obtidos pelo piloto por ano de participação, indicando as corridas em que os pontos foram obtidos.

8.7 Relatório 7 – Piloto: Status das Corridas

Este relatório apresenta a quantidade de resultados por status nas corridas em que o piloto participou.

9 Índices Criados

Os índices foram criados para auxiliar consultas com filtros, junções e ordenações frequentes.

Índice	Justificativa
--------	---------------

<code>idx_results_status</code>	Facilitar consultas que agrupam ou filtram por status de resultado.
<code>idx_results_constructor_driver</code>	Auxiliar consultas que agrupam resultados por escuderia e contam pilotos distintos, especialmente no Relatório Admin 3 e nos relatórios de escuderia.
<code>idx_cities_country_lower_name_coordinates</code>	Auxiliar a busca de cidades brasileiras por nome no Relatório 2, considerando apenas cidades com coordenadas disponíveis.
<code>idx_airports_city_type_coordinates</code>	Auxiliar a seleção de aeroportos com cidade, tipo e coordenadas disponíveis no Relatório 2.
<code>idx_results_driver_race_points_positive</code>	Auxiliar o Relatório 6, filtrando resultados pontuados por piloto e facilitando a junção com corridas.
<code>idx_app_escuderia_pilotos_piloto</code>	Auxiliar consultas que partem de um piloto para encontrar vínculos com escuderias na camada da aplicação.
<code>idx_app_import_pilotos_escuderia_pendentes</code>	Auxiliar o processamento de linhas pendentes carregadas por arquivo para uma escuderia.
<code>idx_drivers_lower_family_name</code>	Auxiliar buscas de pilotos por sobrenome sem diferenciar letras maiúsculas e minúsculas.

10 Interface da Aplicação

A interface foi organizada em três telas principais:

- tela de login;
- tela de *dashboard*;
- tela de relatórios.

10.1 Tela de Login

Inserir descrição e captura de tela.

Figura 1: Tela de login da aplicação.

10.2 Tela de Dashboard

Inserir descrição e captura de tela.

Figura 2: Tela de dashboard.

10.3 Tela de Relatórios

Inserir descrição e captura de tela.

Figura 3: Tela de relatórios.

11 Decisões de Projeto

12 Dificuldades Encontradas

13 Conclusão